

DA3408 AI Operations Assignment 2

Prabhav Gupta DA24B018

Question 1 — Single-Stage vs. Multi-Stage Docker

I built the spam API two ways: a naive Dockerfile and a multi-stage Dockerfile. Both serve `/predict` and `/healthz` the same way. The naive image is **593 MB**. The multi-stage image is **129 MB**. That is a **78.24% reduction**.

The multi-stage image is smaller for two reasons. First, its final stage uses `python:3.10-slim` instead of `python:3.10`. The naive image has `gcc`, `g++` and `make` installed, but the multi-stage image has none of these. Second, `pandas` is missing from the multi-stage image, while the naive image has it. Pandas is only needed to build the training data, not to make predictions, so it is left out of the final image.

Question 2 — Multi-Container Orchestration with Docker Compose

I added a Redis cache to the API. Before predicting, it checks Redis for the same text. On a miss, it computes the answer and saves it. On a hit, it returns the saved answer. `docker-compose.yml` runs the API and Redis together, and the API can reach Redis just by using the name `cache`.

I tested this by sending 200 messages twice. The first time was **200/200 misses**, with a median time of 1.33 ms. The second time was **200/200 hits**, with a median of 0.42 ms. That is about **3.2x faster**.

Compose lets a few containers run together on one machine. It cannot run containers across many machines. Kubernetes can do that, and it also keeps a set number of copies of each container running on its own.

Question 3 — Kubernetes Indexed Job: Parallel Data Validation

I made 8 CSV files of fake signup records. Each file has a different, known number of bad rows. Each pod reads its own job index to know which file to check.

The cluster has 4 CPUs in total. Kubernetes itself uses about 1250m of that, leaving about 2750m free. Each pod only needs 500m, since the validator is single-threaded. Four pods need 2000m, which fits easily. Going higher would not help, since the machine only has 4 cores. So I set **parallelism to 4**. I confirmed with a script that 4 pods really did run at once, and the job finished all 8 shards in 48 seconds.

I collected the results using the Kubernetes API instead of a shared folder, since minikube's storage is tied to one node and would not work if pods land on different nodes. All 8 results

matched what I expected.

With 6 CPUs instead of 4, parallelism should go up to **8**, not just a bit higher. That is because 8 pods only need 4000m of the 6000m available. At parallelism 4, the job needed two rounds to finish. Running all 8 pods at once removes that second round.

Question 4 — Kubernetes Deployments: Self-Healing and Rolling Updates

I deployed the API with 2 replicas and a health check on `/healthz`. Both pods came up correctly, and traffic was split between them.

To test self-healing, I deleted a running pod by hand. A new pod came up within 1–2 seconds. The logs show this was done by the **ReplicaSet**, not the Deployment directly. A ReplicaSet keeps a fixed number of pods running and notices right away if one goes missing.

To test a rolling update, I checked `/healthz` every 0.2 seconds while updating the app from v1 to v2. Out of 607 checks, 604 got a normal response. That is **99.5% uptime**. The 3 that failed all happened at the exact moment the old and new pods were being swapped.

If this had been a Job instead of a Deployment, updating it would mean deleting it and starting over, with no pod running in between. A Job also expects its pods to finish, but this API never stops on its own.

The Job in Question 3 finishes when its pods stop. This Deployment succeeds when its pods keep running, so a pod that stops gets replaced right away. This fits each task well: Question 3 was a fixed amount of work with a clear end, while this API needs to run all the time and be updated without going down.